Pengenalan Framework (Django)

Django merupakan salah satu framework pengembangan aplikasi web berbasis Python yang memiliki banyak keunggulan, antara lain:

- **Secure**: menyediakan beberapa fitur keamanan untuk mencegah SQL injenction, clickjacking, cross site scripting dan lain sebagainya.
- **Full Battery Included**: secara bawaan django sudah menyediakan fitur aplikasi web yang cukup lengkap seperti templating, ORM, authentication service, dan juga web admin.
- **Dokumentasi lengkap**: dokumentasi dan userguide yang tersedia cukup jelas dan sudah dapat membantu pembelajaran bagi orang awam yang belum pernah menggunakannya.
- **Dukungan komunitas**: komunitas pengembang cukup solid, sehingga akan mudah menemukan dokumentasi, tutorial, dan jawaban pertanyaan umum. Selain itu juga terdapat banyak plugin untuk menyediakan fitur yang belum tercantum pada bawaan framework.

Penggunaan framework Django juga memiliki beberapa catatan antara lain:

- Bahasa pemrograman yang digunakan adalah Python.
- Deployment aplikasi tidak dapat menggunakan sharedhost biasa, kita harus memastikan hosting yang digunakan dapat support django.
- Pendekatan yang digunakan adalah modular application, di mana setiap modul memiliki struktur MVC-nya sendiri.
- Support bawaan untuk frontend relatif lemah. Django hanya menyediakan default template engine, sedangkan support untuk framework frontend seperti React & Vue dapat dilakukan secara terpisah atau menggunakan library pihak ketiga.

Boostraping Django

Secara sederhana kita dapat memulai django dengan berikut:

- Install python
- Buat virtual environment `python -m venv env_django`
- Aktifkan virtual environment:
  - Windows: `.\env_django\Scripts\activate`
  - Mac & Linux: `source ./env_django/bin/activate`
- Install django dengan `pip install django`
- Buat project dengan perintah `python -m django startproject simplelms`
- Buat (modul) aplikasi dengan perintah `python manage.py startapp core`
- Buat basis data dengan perintah `python manage.py migrate`
- Jalankan aplikasi dengan `python manage.py runserver`

Namun jika kita ingin setup aplikasi Django dari awal dengan model containerisasi yang sesuai, maka cara bootstraping berikut dapat digunakan.

Boostraping Docker Django

Proses *bootstraping* ini agak panjang karena kita benar-benar memulai project dari nol, tanpa source code django, tanpa library pendukung, bahkan tanpa python. Setelah menyelesaikan proses bootstraping ini, kita sudah memiliki template aplikasi django yang bisa dikembangkan sesuka hati.

- Pertama buat `Dockerfile` dan isikan dengan koding berikut

```
FROM python:latest
ENV PYTHONUNBUFFERED=1
WORKDIR /code
COPY requirements.txt /code/
RUN pip install -r requirements.txt
CMD ["sleep", "infinity"]
```

- Selanjutnya buat file `code/requirements.txt` dan isikan dengan koding berikut

```
django
psycopg2-binary
```

- Selanjutnya, buat file `docker-compose.yml` dan isikan dengan koding berikut

```
services:
  django:
    container_name: simple_lms
    build: .
    command: sleep infinity
    # command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ./code:/code
    ports:
      - "8001:8000"
  postgres:
    container_name: simple_db
    image: postgres:latest
    environment:
      POSTGRES_DB: simple_lms
      POSTGRES_USER: simple_user
      POSTGRES_PASSWORD: simple_password
      PGDATA: /var/lib/postgresql/data/pgdata
    ports:
      - "5532:5432"
    volumes:
      - ./postgres-data:/var/lib/postgresql/data/pgdata
```

- Jalankan perintah untuk build dan run container

```
docker compose up -d --build
```

- Jalankan perintah berikut untuk membuat project simplelms pada direktori code

```
docker exec simple_lms django-admin startproject simplelms .
docker exec simple_lms python manage.py startapp core
```

- Buat file baru bernama `simplelms/local_settings.py` dan isikan dengan koding berikut:

```
from django.conf import settings

DEBUG = True
ALLOWED_HOSTS = ['*']

settings.INSTALLED_APPS += [
    'core',
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql_psycopg2',
        'NAME': 'simple_lms',
        'USER': 'simple_user',
        'PASSWORD': 'simple_password',
        'HOST': 'simple_db',
    }
}
```

- Tambahkan koding berikut di bagian akhir file `simplelms/settings.py`

```
try:
    from simplelms.local_settings import *
except:
    pass
```

- Uji coba aplikasi dengan menjalankan perintah

```
docker exec simple_lms python manage.py runserver 0.0.0.0:8000
```

- Untuk melakukan migrasi basis data default jalankan perintah berikut

```
docker exec simple_lms python manage.py makemigrations
docker exec simple_lms python manage.py migrate
```

- Ubah property `command` pada file `docker-compose.yml` dari `sleep infinity` menjadi `python manage.py runserver 0.0.0.0:8000`
- Setelah selesai melakukan perubahan, kita perlu melakukan `down` dan `up` pada container kita dengan perintah

```
docker-compose down
docker-compose up -d
```

Tambahan:

- Jika kita ingin membuat superuser default pada aplikasi, kita bisa menggunakan perintah

```
docker exec -it simple_lms python manage.py createsuperuser
```